IT Ticket Manager

Этап 1. Планирование настольного приложения

Индивидуальная работа по дисциплине
Planificarea aplicațiilor Desktop

Вариант №10 • Группа P2524r
Студент: Natalia Gairunova
Преподаватель: Natalia Gairunova

2026

Содержание этапа

- 1. Общая характеристика приложения
- 2. Актуальность, цель и задачи проекта
- 3. Границы проекта и ожидаемый результат
- 4. Модули приложения
- 5. Архитектура и взаимодействие компонентов
- 6. Пользователи и роли
- 7. Данные и жизненный цикл заявки
- 8. Основные сценарии использования
- 9. Функциональные и нефункциональные требования
- 10. Технологические решения
- 11. План выполнения на 40 дней
- 12. Ограничения, риски и допущения
- 13. Итог этапа 1

Примечание. Этот документ описывает проектирование и планирование будущего приложения. На момент подготовки этапа программная реализация, подключение базы данных, отправка писем и публикация прототипа не считаются выполненными результатами.

1. Общая характеристика приложения

1.1. Название и назначение

IT Ticket Manager — проект настольного приложения для регистрации, обработки и контроля технических заявок. Приложение предназначено для небольшой организации, колледжа или учебной лаборатории, где пользователи обращаются к IT-специалисту по вопросам компьютеров, программного обеспечения, сети и периферийного оборудования.

Система должна заменить неформальную передачу проблем через устные сообщения или разрозненные переписки единым управляемым процессом. Каждая заявка получает номер, описание, приоритет, автора, ответственного техника и текущий статус. Такой подход создаёт понятную историю работы и помогает определить, какие обращения требуют внимания в первую очередь.

1.2. Рабочая модель предметной области

Пользователь запускает приложение, проходит авторизацию и создаёт заявку. В заявке указываются краткий заголовок, подробное описание, категория проблемы и приоритет. Техник просматривает очередь, выбирает обращение, принимает его в работу, фиксирует результат и переводит заявку в состояние «Решена». После подтверждения результата заявка может быть закрыта.

Элемент	Описание
Заявка	Запись о конкретной технической проблеме с номером, описанием, приоритетом и историей изменений.
Пользователь	Автор обращения, который создаёт заявку и отслеживает её состояние.
Техник	Сотрудник, который анализирует заявку, выполняет работу и фиксирует результат.
Статус	Состояние заявки в процессе обработки: Открыта, В работе, Решена, Закрыта.
Уведомление	Информационное сообщение о создании заявки или изменении её статуса; в проекте предусмотрено как планируемая функция.

1.3. Принцип работы

- Заявка создаётся пользователем и получает уникальный идентификатор.
- Техник видит новые обращения в рабочем списке и оценивает их приоритет.
- После принятия заявки в работу система сохраняет ответственного исполнителя и время изменения.
- Результат решения записывается в карточке заявки; после проверки заявка закрывается.
- Изменения статуса и ключевые события должны быть доступны в истории заявки.

2. Актуальность, цель и задачи проекта

2.1. Актуальность

В небольшой организации технические проблемы часто сообщаются напрямую специалисту. При увеличении числа обращений такой способ становится ненадёжным: часть проблем теряется, сроки не фиксируются, а пользователь не понимает, кто занимается его вопросом. IT Ticket Manager предлагает простой и проверяемый процесс, в котором каждое обращение имеет состояние и историю.

2.2. Цель проекта

Цель проекта — спроектировать настольное приложение, обеспечивающее удобную регистрацию, обработку и контроль выполнения IT-заявок, а также подготовить согласованный комплект проектной документации для последующей реализации.

2.3. Задачи проекта

- определить пользователей, роли и границы системы;
- описать жизненный цикл заявки и допустимые переходы между статусами;
- разделить систему на понятные функциональные модули;
- сформировать логическую архитектуру и структуру данных;
- подготовить функциональные и нефункциональные требования;
- определить технологический стек для возможной реализации;
- спроектировать сценарии, которые будут отражены в UML и прототипе Figma;
- составить реалистичный план работ на 40 дней.

2.4. Ожидаемая польза

Для пользователя польза заключается в понятной регистрации обращения и возможности видеть его состояние. Для техника — в единой очереди, приоритизации и сохранении истории. Для организации — в прозрачности процесса и возможности анализировать повторяющиеся проблемы. Эти эффекты являются ожидаемыми результатами проектируемой системы, а не утверждением о уже полученной эксплуатации.

3. Границы проекта и ожидаемый результат

3.1. Что входит в проектирование

Входит в этап 1	Статус
Описание назначения и предметной области	Планируется и документируется
Роли, права и сценарии	Планируется и документируется
Модули и архитектура	Проектируется
Структура данных	Проектируется логически
Требования к будущей системе	Формулируются
UML и Figma	Определяются как следующие проектные результаты
Программный код и рабочая база данных	Не входят в текущий этап
Реальная отправка email	Не реализуется на текущем этапе

3.2. Что не утверждается как выполненное

В рамках индивидуальной работы создаётся план и комплект проектных материалов. Поэтому формулировки «система должна», «предполагается», «будет подготовлено» относятся к проектным решениям. Не следует писать, что приложение уже принимает заявки, отправляет письма или хранит реальные данные, если это не подтверждено отдельным результатом реализации.

3.3. Критерии готовности этапа 1

- назначение и границы IT Ticket Manager сформулированы однозначно;
- описаны модули, роли, данные и жизненный цикл заявки;
- сценарии согласованы с предполагаемыми функциями;

- требования можно связать с будущими UML-диаграммами и прототипом;
- выбран технологический подход и зафиксированы его ограничения;
- подготовлен календарный план на 40 дней.

4. Модули приложения

Модульная структура используется как средство декомпозиции проекта. Она не означает, что все модули уже реализованы отдельными программными компонентами.

ID	Модуль	Назначение	Основные входные данные	Ожидаемый результат
M1	Авторизация и сессия	Вход по учётным данным, выход из системы, определение роли.	Логин, пароль; профиль пользователя.	Успешный вход или сообщение об ошибке.
M2	Пользователи и роли	Хранение профилей и разграничение доступа.	Имя, email, роль, признак активности.	Разрешённые действия в интерфейсе.
M3	Создание заявки	Ввод и первичная проверка обращения.	Заголовок, описание, категория, приоритет.	Новая заявка со статусом «Открыта».
M4	Очередь заявок	Поиск, фильтрация и просмотр заявок техником.	Статус, приоритет, дата, категория, исполнитель.	Список, отсортированный по заданным условиям.
M5	Обработка заявки	Назначение, смена статуса и фиксация результата.	Номер заявки, комментарий, новый статус.	Обновлённая карточка и запись в истории.
M6	История и аудит	Хронология изменений заявки.	Событие, автор, дата, старое и новое значение.	Последовательная история действий.
M7	Уведомления	Подготовка событий для email-уведомлений.	Получатель, тип события, текст, статус отправки.	Запланированное уведомление; фактическая доставка требует реализации.

4.1. Связи между модулями

Авторизация определяет роль и передаёт контекст пользователя остальным модулям. Модуль создания заявки формирует запись, которую затем использует очередь заявок. Обработка заявки изменяет её статус и одновременно создаёт событие в истории. Модуль уведомлений получает сведения об этих событиях, но способ реальной отправки писем относится к будущей реализации.

5. Архитектура и взаимодействие компонентов

5.1. Выбранный архитектурный подход

Для будущей реализации предлагается слоистая архитектура с разделением интерфейса, прикладной логики и доступа к данным. Такой подход облегчает тестирование и позволяет менять отдельные технические решения без переписывания всей системы.

Слой	Ответственность	Примеры компонентов
Представление	Окна, формы, таблицы, фильтры, сообщения об ошибках.	LoginView, DashboardView, TicketForm, TicketDetailsView
Прикладная логика	Проверка правил, сценарии создания и обработки заявки.	AuthService, TicketService, NotificationService
Модель домена	Сущности и допустимые состояния предметной области.	User, Ticket, TicketStatus, TicketHistory
Доступ к данным	Чтение и сохранение пользователей,	UserRepository, TicketRepository,

	заявок и истории.	HistoryRepository
Инфраструктура	SQLite, хеширование паролей, почтовый шлюз при необходимости.	DatabaseProvider, MailGateway

5.2. Логика взаимодействия

- Пользователь выполняет действие в окне приложения.
- Представление передаёт данные прикладному сервису.
- Сервис проверяет права и бизнес-правила.
- Репозиторий сохраняет или извлекает данные.
- Результат возвращается в интерфейс; при значимом событии формируется запись истории и, при будущей реализации, уведомление.

5.3. Доступ и безопасность

Пароли не должны храниться в открытом виде; в проекте предусматривается хранение хеша. Операции техника должны быть недоступны обычному пользователю. При ошибке авторизации приложение не должно раскрывать лишние сведения о существовании учётной записи. Конкретный алгоритм и политика хранения секретов будут уточнены на этапе реализации.

6. Пользователи и роли

Роль	Цель	Разрешённые действия	Ограничения
Пользователь	Сообщить о проблеме и узнать результат.	Войти, создать заявку, просматривать свои заявки, видеть статус, добавить уточнение.	Не изменяет заявки других пользователей и не назначает исполнителей.
Техник	Обработать очередь и решить проблему.	Просматривать доступные заявки, принимать заявку, менять статус, добавлять результат, закрывать заявку.	Не управляет системными настройками, если такая роль не предусмотрена отдельно.
Администратор (опционально)	Поддерживать справочники и учётные записи.	Управлять пользователями, ролями, категориями и общими настройками.	Роль может быть исключена из минимальной версии, если преподаватель требует только две основные роли.

Минимальная версия проекта использует две обязательные роли — пользователь и техник. Администратор описан как возможное расширение, но его наличие не следует считать обязательным требованием базового варианта.

7. Данные и жизненный цикл заявки

7.1. Логическая структура данных

Сущность	Ключевые поля	Назначение
User	user_id, full_name, email, password_hash, role, is_active	Профиль и права пользователя.
Ticket	ticket_id, title, description, category, priority, status, author_id, assignee_id, created_at, updated_at, resolved_at	Основная запись обращения.
TicketHistory	history_id, ticket_id, actor_id, event_type, old_value, new_value, created_at,	История изменений и действий.

	comment	
Notification	notification_id, ticket_id, recipient_id, event_type, delivery_status, created_at	Сведения о планируемом или выполненном уведомлении.

7.2. Статусы и переходы

Текущий статус	Действие	Новый статус	Кто выполняет
—	Создать заявку	Открыта	Пользователь
Открыта	Принять в работу	В работе	Техник
В работе	Зафиксировать решение	Решена	Техник
Решена	Подтвердить завершение	Закрыта	Техник или пользователь по выбранному правилу
В работе	Вернуть на уточнение	Открыта	Техник
Решена	Обнаружить, что проблема не устранена	В работе	Техник

Для учебного проекта рекомендуется явно показать допустимые переходы на Activity Diagram. Возврат из «Решена» в «В работе» нужен, чтобы модель учитывала ситуацию, когда первоначальное решение оказалось недостаточным.

8. Основные сценарии использования

8.1. Создание заявки

Предусловие: пользователь вошёл в приложение. Основной поток: пользователь открывает форму, вводит заголовок и описание, выбирает категорию и приоритет, затем отправляет форму. Система проверяет обязательные поля, создаёт заявку со статусом «Открыта», показывает номер заявки и подготавливает событие для уведомления. Альтернативный поток: если обязательное поле не заполнено, заявка не создаётся и пользователь получает понятное сообщение.

8.2. Обработка заявки техником

Предусловие: техник авторизован. Техник открывает очередь, фильтрует заявки, выбирает обращение и принимает его в работу. Система фиксирует исполнителя и изменяет статус на «В работе». После решения техник добавляет комментарий, переводит заявку в «Решена» и сохраняет событие в истории.

8.3. Закрытие заявки

Пользователь или техник проверяет описание результата. Если проблема устранена, заявка переводится в «Закрыта». Если решение не помогло, заявка возвращается в «В работе» с дополнительным комментарием. Конкретное право пользователя подтверждать закрытие следует закрепить в финальной версии требований; в базовом варианте это может выполнять техник.

8.4. Просмотр истории

Из карточки заявки пользователь видит основные сведения и текущий статус. Техник дополнительно видит последовательность действий: создание, назначение, изменения статуса и комментарии. История нужна для прозрачности, но не должна позволять незаметно изменять прошлые события.

9. Функциональные и нефункциональные требования

9.1. Функциональные требования

ID	Область	Требование
FR-01	Авторизация	Система должна позволять зарегистрированному пользователю войти по учётным данным.
FR-02	Разграничение доступа	Система должна показывать действия в соответствии с ролью пользователя.
FR-03	Создание заявки	Пользователь должен иметь возможность создать заявку с заголовком, описанием, категорией и приоритетом.
FR-04	Идентификатор	Каждой созданной заявке должен назначаться уникальный номер.
FR-05	Очередь	Техник должен иметь возможность просматривать и фильтровать заявки.
FR-06	Назначение	Техник должен иметь возможность принять заявку в работу.
FR-07	Статусы	Система должна поддерживать статусы Открыта, В работе, Решена и Закрыта.
FR-08	История	Система должна сохранять ключевые изменения заявки с автором и временем.
FR-09	Уведомления	Проект должен предусматривать события для уведомлений о создании и изменении статуса; реальная доставка является задачей реализации.
FR-10	Поиск	Техник должен иметь возможность найти заявку по номеру, статусу, категории или приоритету.

9.2. Нефункциональные требования

Категория	Проектное требование
Удобство	Основные действия должны быть доступны из понятных окон; ошибки формы должны объяснять, что нужно исправить.
Производительность	Операции открытия списка и карточки не должны создавать заметной задержки на типичном локальном наборе данных.
Надёжность	Сохранение заявки должно быть атомарным: при ошибке данные не должны оставаться в частично записанном состоянии.
Безопасность	Пароли хранятся в виде хеша; права проверяются до выполнения защищённого действия.
Сопровождаемость	Слои и модули должны иметь отдельные зоны ответственности и понятные интерфейсы.
Переносимость	Решение должно быть рассчитано на запуск на поддерживаемой настольной ОС; конкретный список ОС уточняется до реализации.

10. Технологические решения

Так как реализация программы в текущей работе не требуется, технологический стек рассматривается как обоснованное предложение для будущей разработки, а не как отчёт о реально использованных инструментах.

Компонент	Предлагаемое решение	Причина выбора
Язык	Java	Изучается автором и подходит для объектно-ориентированного настольного приложения.
Интерфейс	JavaFX	Предоставляет средства для окон, форм, таблиц, сцен и стилизации.
Локальная БД	SQLite	Подходит для небольшого автономного приложения без отдельного серверного процесса.
Доступ к БД	JDBC или совместимый слой доступа	Позволяет отделить SQL-запросы от прикладной логики.
Письма	SMTP через отдельный сервисный слой	Позволяет заменить способ доставки без изменения сценариев заявок.
Документация	Word/PDF, PlantUML, Figma	Соответствует планируемым материалам индивидуальной работы.
Версионирование	Git и GitHub	Фиксирует историю проектных материалов и упрощает проверку изменений.

10.1. Обоснование отказа от усложнения

Для минимальной версии не требуется серверная архитектура, распределённая база данных или мобильный клиент. Такие решения увеличили бы объём реализации и не нужны для демонстрации базового жизненного цикла заявки. Расширение до многопользовательской сетевой системы можно рассматривать после подтверждения требований и ограничений проекта.

11. План выполнения на 40 дней

Период	Работы	Результат
Дни 1–3	Уточнение задания, границ и критериев оценки.	Утверждённые параметры проекта.
Дни 4–7	Описание предметной области, целей и ролей.	Разделы 1–3 отчёта.
Дни 8–11	Проектирование модулей и архитектуры.	Модульная схема и описание слоёв.
Дни 12–15	Модель данных и статусы заявки.	Таблица сущностей и переходов.
Дни 16–20	Функциональные и нефункциональные требования.	Черновик SRS.
Дни 21–25	Use Case и Activity диаграммы, проверка соответствия SRS.	Изображения и код PlantUML.
Дни 26–31	Прототип Figma и mini-card жизненного цикла заявки.	Редактируемый макет и скриншоты.
Дни 32–35	Структура GitHub, README, описание команд Git.	Организованный репозиторий.
Дни 36–38	Сборка отчёта и проверка ссылок, нумерации и формулировок.	Предварительная версия отчёта.
Дни 39–40	Финальная проверка и подготовка к	Итоговый комплект материалов.

	защите.	

12. Ограничения, риски и допущения

Риск или ограничение	Влияние	Мера предупреждения
Нет реализации программы	Нельзя подтверждать работу функций реальными тестами.	Использовать проектные формулировки и отдельно описать планируемые результаты.
Не уточнены требования к ОС	Может потребоваться изменение технических деталей.	Зафиксировать целевую ОС до начала возможной реализации.
Email зависит от внешнего SMTP	Доставка может требовать учетных данных и сетевого доступа.	Описать интерфейс уведомлений отдельно от транспорта; в прототипе показать событие, а не факт доставки.
Две роли могут оказаться недостаточными	Для администрирования потребуется расширение модели.	Оставить администратора как опциональную роль и не смешивать её с базовым сценарием.
Избыточный объём документа	Сложнее проверить согласованность материалов.	Связывать каждый раздел с целью проекта, требованиями и будущими диаграммами.

12.1. Основные допущения

- приложение предназначено для небольшой локальной организации;
- минимальная версия использует локальное хранение данных;
- пользователь и техник являются обязательными ролями;
- статусы и переходы могут быть уточнены после обратной связи преподавателя;
- email-уведомления являются частью проектируемого поведения, но не доказательством выполненной интеграции.

13. Итог этапа 1

В результате этапа 1 сформирован целостный план настольного приложения IT Ticket Manager. Определены назначение системы, её границы, роли пользователей, модули, архитектурный подход, логическая модель данных, жизненный цикл заявки, сценарии, требования и предлагаемый технологический стек. Также составлен календарный план на 40 дней и зафиксированы ограничения, которые важно учитывать при дальнейшей подготовке работы.

Следующим логичным этапом является детализация SRS и подготовка двух согласованных UML-диаграмм: Use Case Diagram для ролей и возможностей системы и Activity Diagram для обработки заявки. После этого можно разработать прототип Figma и включить его материалы в итоговый отчёт. Все последующие результаты должны оставаться согласованными с данным планом.

Приложение А. Краткий словарь терминов

Термин	Значение
Ticket / заявка	Запись о технической проблеме пользователя.
Приоритет	Оценка срочности обработки обращения.
Статус	Текущее состояние заявки в жизненном цикле.
Техник	Пользователь с правом обработки заявок.

SRS	Software Requirements Specification — спецификация требований к программному обеспечению.
UML	Унифицированный язык моделирования программных систем.